

Intl.NumberFormat

Baseline Widely available

The `Intl.NumberFormat` object enables language-sensitive number formatting.

Try it

JavaScript Demo: Intl.NumberFormat

```
1 const number = 123456.789;
2
3 console.log(
4   new Intl.NumberFormat('de-DE', { style: 'currency', currency: 'EUR' }).format(
5     number,
6   ),
7 );
8 // Expected output: "123.456,79 €"
9
10 // The Japanese yen doesn't use a minor unit
11 console.log(
12   new Intl.NumberFormat('ja-JP', { style: 'currency', currency: 'JPY' }).format(
13     number,
14   ),
15 );
16 // Expected output: "¥123,457"
17
18 // Limit to three significant digits
19 console.log(
20   new Intl.NumberFormat('en-IN', { maximumSignificantDigits: 3 }).format(
21     number,
22   ),
23 );
24 // Expected output: "1,23,000"
--
```

Constructor

[Intl.NumberFormat\(\)](#)

Creates a new `NumberFormat` object.

Static methods

[Intl.NumberFormat.supportedLocalesOf\(\)](#)

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.NumberFormat.prototype` and shared by all `Intl.NumberFormat` instances.

[Intl.NumberFormat.prototype.constructor](#)

The constructor function that created the instance object. For `Intl.NumberFormat` instances, the initial value is the [Intl.NumberFormat](#) constructor.

`Intl.NumberFormat.prototype[Symbol.toStringTag]`

The initial value of the [\[Symbol.toStringTag\]](#) property is the string `"Intl.NumberFormat"`. This property is used in [Object.prototype.toString\(\)](#).

Instance methods

[Intl.NumberFormat.prototype.format\(\)](#)

Getter function that formats a number according to the locale and formatting options of this `Intl.NumberFormat` object.

[Intl.NumberFormat.prototype.formatRange\(\)](#)

Getter function that formats a range of numbers according to the locale and formatting options of the `Intl.NumberFormat` object from which the method is called.

[Intl.NumberFormat.prototype.formatRangeToParts\(\)](#)

Returns an [Array](#) of objects representing the range of number strings in parts that can be used for custom locale-aware formatting.

[Intl.NumberFormat.prototype.formatToParts\(\)](#)

Returns an [Array](#) of objects representing the number string in parts that can be used for custom locale-aware formatting.

[Intl.NumberFormat.prototype.resolvedOptions\(\)](#)

Returns a new object with properties reflecting the locale and collation options computed during initialization of the object.

Examples

Basic usage

In basic use without specifying a locale, a formatted string in the default locale and with default options is returned.

JS

```
const number = 3500;

console.log(new Intl.NumberFormat().format(number));
// '3,500' if in US English locale
```

Using locales

This example shows some of the variations in localized number formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

JS

```
const number = 123456.789;

// German uses comma as decimal separator and period for thousands
console.log(new Intl.NumberFormat("de-DE").format(number));
// 123.456,789

// Arabic in most Arabic speaking countries uses real Arabic digits
console.log(new Intl.NumberFormat("ar-EG").format(number));
// ١٢٣٤٥٦,٧٨٩

// India uses thousands/lakh/crore separators
console.log(new Intl.NumberFormat("en-IN").format(number));
// 1,23,456.789

// the nu extension key requests a numbering system, e.g. Chinese decimal
console.log(new Intl.NumberFormat("zh-Hans-CN-u-nu-hanidec").format(number));
// 一二三,四五六.七八九

// when requesting a language that may not be supported, such as
```

```
// Balinese, include a fallback language, in this case Indonesian
console.log(new Intl.NumberFormat(["ban", "id"]).format(number));
// 123.456,789
```

Using options

The results can be customized using the [options](#) argument:

```
JS

const number = 123456.789;

// request a currency format
console.log(
  new Intl.NumberFormat("de-DE", { style: "currency", currency: "EUR" }).format(
    number,
  ),
);
// 123.456,79 €

// the Japanese yen doesn't use a minor unit
console.log(
  new Intl.NumberFormat("ja-JP", { style: "currency", currency: "JPY" }).format(
    number,
  ),
);
// ¥123,457

// limit to three significant digits
console.log(
  new Intl.NumberFormat("en-IN", { maximumSignificantDigits: 3 }).format(
    number,
  ),
);
// 1,23,000

// Formatting with units
console.log(
  new Intl.NumberFormat("pt-PT", {
    style: "unit",
    unit: "kilometer-per-hour",
  }).format(50),
);
// 50 km/h

console.log(
  (16).toLocaleString("en-GB", {
    style: "unit",
    unit: "liter",
    unitDisplay: "long",
  }),
);
// 16 litres
```

For an exhaustive list of options, see the [Intl.NumberFormat\(\) constructor](#) page.

Specifications

Specification
ECMAScript Internationalization API Specification # numberformat-objects

Browser compatibility

[Report problems with this compatibility data on GitHub](#)

	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Internet
NumberFormat	24	12	29	15	10	25	56	14	10	1.5
NumberFormat(). constructor	24	12	29	15	10	25	56	14	10	1.5
Supports normative optional ChainNumberFormat behavior	91	91	54	77	14.1	91	56	64	14.5	16.0
locales parameter	24	12	29	15	10	25	56	14	10	1.5
options parameter	24	12	29	15	10	25	56	14	10	1.5
options.compactDisplay parameter	77	79	78	64	14.1	77	79	55	14.5	12.0
options.currencyDisplay parameter	77	79	78	64	14.1	77	79	55	14.5	12.0
options.currencySign parameter	77	79	78	64	14.1	77	79	55	14.5	12.0
options.currency parameter	24	12	29	15	10	25	56	14	10	1.5
options.localeMatcher parameter	24	12	29	15	10	25	56	14	10	1.5
options.maximumFractionDigits parameter	24	12	29	15	10	25	56	14	10	1.5
options.maximumSignificantDigits parameter	24	12	29	15	10	25	56	14	10	1.5
options.minimumFractionDigits parameter	24	12	29	15	10	25	56	14	10	1.5
options.minimumIntegerDigits parameter	24	12	29	15	10	25	56	14	10	1.5
options.minimumSignificantDigits parameter	24	12	29	15	10	25	56	14	10	1.5
options.notation parameter	77	79	78	64	14.1	77	79	55	14.5	12.0
options.numberingSystem parameter	24	12	29	15	10	25	56	14	10	1.5
options.roundingIncrement parameter	106	106	116	92	15.4	106	116	72	15.4	20.0
options.roundingMode parameter	106	106	116	92	15.4	106	116	72	15.4	20.0
options.roundingPriority parameter	106	106	116	92	15.4	106	116	72	15.4	20.0
options.signDisplay parameter	77	79	78	64	14.1	77	79	55	14.5	12.0
negative value	106	106	116	92	15.4	106	116	72	15.4	20.0
options.style parameter	24	12	29	15	10	25	56	14	10	1.5
options.trailingZeroDisplay parameter	106	106	116	92	15.4	106	116	72	15.4	20.0

		Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Internet
	options.unitDisplay parameter	77	79	78	64	14.1	77	79	55	14.5	12.0
	options.unit parameter	77	79	78	64	14.1	77	79	55	14.5	12.0
	options.useGrouping parameter	24	12	29	15	10	25	56	14	10	1.5
options.useGrouping parameter accepts: 'always', 'auto', 'min2' (in addition to: true and false)		106	106	116	92	16.4	106	116	72	16.4	20.0
format		24	12	29	15	10	25	56	14	10	1.5
number param string value is decimal (not Number)		106	106	116	92	15.4	106	116	72	15.4	20.0
formatRange		106	106	116	92	15.4	106	116	72	15.4	20.0
formatRangeToParts		106	106	116	92	15.4	106	116	72	15.4	20.0
formatToParts		64	12	58	51	13	64	58	47	13	9.0
resolvedOptions		24	12	29	15	10	25	56	14	10	1.5
supportedLocalesOf		24	12	29	15	10	25	56	14	10	1.5

Tip: you can click/tap on a cell for more information.

Full support Partial support No support Deprecated. Not for use in new websites. See implementation notes.

Has more compatibility info.

See also

- [Polyfill of Intl.NumberFormat in FormatJS](#)

Help improve MDN

Was this page helpful to you?

Yes

No

[Learn how to contribute.](#)

This page was last modified on Jul 24, 2024 by [MDN contributors](#).